

Práctica N.º 2

Navegación, listas, formularios y manejo del tiempo

Asignatura	Desarrollo de Aplicaciones Móviles (INF 662)	•	6.º semestre
Docente	M. Sc. Lic. Huáscar Fedor Gonzales Guzmán		
Base	Flutter (canal estable) • Dart 3 • solo SDK, sin paquetes externos		
Modalidad	Individual (trabajo independiente)	•	Fecha de entrega: 14 / 09 / 2026

En esta práctica integrarás lo trabajado en las Guías de Laboratorio 1 a 8 de la asignatura: la construcción de interfaces, la navegación con paso de datos entre pantallas, las listas y cuadrículas, los formularios con validación, el manejo del estado con `setState` y el uso del temporizador. Desarrollarás de forma individual las tres aplicaciones que se describen a continuación.

01 Objetivos

- Navegar entre pantallas y **pasar datos** por constructor con `Navigator.push` / `pop` y `MaterialPageRoute`.
- Mostrar colecciones con `ListView.builder` y `GridView.builder` usando `Card` y un modelo de datos propio.
- Construir formularios con `Form` y `TextFormField`, con validación y manejo seguro de valores numéricos.
- Gestionar el estado con `setState` y liberar recursos en `dispose`.
- Emplear `Timer.periodic` para lógica basada en el tiempo, con su correcta cancelación.

02 Contenidos que se evalúan

La práctica se limita a los siguientes contenidos. No utilices Provider, sqflite, ni bases de datos externas: se abordarán en prácticas posteriores. Los datos viven en memoria.

ALCANCE

Interfaz y navegación: Scaffold, AppBar, Column, Row, Container, Card, `ListView.builder`, `GridView.builder`, `Navigator.push` / `pop`, `MaterialPageRoute` (paso de datos por constructor).

Datos y estado: `StatefulWidget` + `setState`, modelo de datos propio, `Form` / `TextFormField` con validación, `TextEditingController` (con `dispose`), `double.tryParse` / `int.tryParse`.

Tiempo: `Timer.periodic` de `dart:async`, con cancelación en `dispose`.

03 Instrucciones generales

- Trabajo individual. Cada estudiante entrega su propia solución; el código copiado invalida la calificación de las partes involucradas.

- Un proyecto Flutter por aplicación (o un repositorio con una carpeta por app, nombradas `app1_ecoguia`, `app2_gastodia`, `app3_quizrelampago`).
- Cada aplicación usa al menos un `StatefulWidget` y actualiza su interfaz mediante `setState`.
- Solo se permite el SDK de Flutter; no agregues paquetes externos.
- Formatea el código con `dart format` y usa nombres de variables y widgets descriptivos.
- Incluye por cada app una captura de pantalla y un breve README (qué hace y cómo ejecutarla).

04 Aplicaciones a desarrollar

Desarrolla las tres aplicaciones. Cada una vale 30 puntos (ver criterios de evaluación). Los retos opcionales otorgan puntaje adicional (bonus) hasta un máximo de 10 puntos sobre el total.

APP 1 EcoGuía — Catálogo de especies con ficha de detalle

Objetivo. Practicar la navegación con paso de datos entre pantallas y el uso de cuadrículas con un modelo propio.

REQUISITOS MÍNIMOS

- Un **modelo de datos** propio (p. ej. clase `Especie` con nombre, categoría, descripción y datos curiosos).
- Pantalla principal con `GridView.builder` de `Card` (al menos 8 elementos): imagen o ícono, nombre y categoría.
- Al tocar una tarjeta, navegar a la ficha pasando el objeto **por el constructor** con `MaterialPageRoute`.
- La ficha muestra la descripción y los datos curiosos con un `ListView`.
- Marcar como favorita y **devolver ese dato** a la lista con `Navigator.pop`; confirmar con un `SnackBar`.

Widgets clave

`GRIDVIEW.BUILDER`

`CARD`

`MODELO DE DATOS`

`NAVIGATOR.PUSH/POP`

`MATERIALPAGEROUTE`

`LISTVIEW`

`SNACKBAR`

RETO OPCIONAL (BONUS)

Agrega un buscador (`TextField` + `setState`) que filtre la cuadrícula por nombre mientras se escribe.

APP 2 GastoDía — Registro de gastos con formulario validado

Objetivo. Construir un formulario con validación y gestionar una colección en memoria con `setState`.

REQUISITOS MÍNIMOS

- Formulario con `Form` + `GlobalKey<FormState>`, `TextFormField` para monto y descripción, y `DropdownButtonFormField` para la categoría.
- Validaciones: monto numérico **mayor que cero** (`double.tryParse`), descripción no vacía y categoría obligatoria.

- Al validar con `validate()`, agregar el gasto con `setState`, limpiar el formulario y liberar los controladores en `dispose`.
- Lista con `ListView.builder` (`CircleAvatar` por categoría), **total acumulado** visible y opción de eliminar un gasto.
- Manejar el **estado vacío** y navegar al detalle de un gasto pasando los datos por constructor.

Widgets clave `FORM` `TEXTFORMFIELD` `GLOBALKEY<FORMSTATE>` `VALIDATE()` `DROPDOWNBUTTONFORMFIELD`
`LISTVIEW.BUILDER` `DISPOSE`

ADVERTENCIA

El monto debe tolerar entradas vacías o no numéricas sin cerrarse ni mostrar **NaN**.

RETO OPCIONAL (BONUS)

Muestra un **resumen por categoría** (cuánto se gastó en cada una) o un filtro por categoría con `DropDownButton`.

APP 3

QuizRelámpago — Cuestionario con temporizador

Objetivo. Integrar un temporizador de cuenta regresiva con navegación y actualización condicional de la interfaz.

REQUISITOS MÍNIMOS

- Un **banco de al menos 6 preguntas** (modelo propio o `List` de objetos) con enunciado, opciones y respuesta correcta.
- Pantalla de inicio con un botón **Comenzar** que navega al cuestionario.
- En cada pregunta, un `Timer.periodic` que descuenta el tiempo (p. ej. 10 s) y actualiza la interfaz con `setState`; al llegar a cero, avanza.
- Registrar el puntaje y, al terminar, navegar a una pantalla de resultados pasando el puntaje **por constructor**.
- Cancelar el `Timer` en `dispose` y al cambiar de pregunta, para evitar temporizadores acumulados.

Widgets clave `TIMER.PERIODIC` `DART:ASYNC` `SETSTATE` `DISPOSE` `NAVIGATOR + CONSTRUCTOR` `LIST`
 / `MODELO`

RETO OPCIONAL (BONUS)

Resalta en verde/rojo la opción correcta e incorrecta antes de avanzar y agrega una barra de progreso (`LinearProgressIndicator`) del tiempo restante.

05 Producto entregable

- Código fuente de las tres aplicaciones (proyectos Flutter o carpetas dentro de un repositorio).
- Una captura de pantalla por aplicación mostrando su funcionamiento (lista/cuadrícula, detalle, formulario, resultados, etc.).

- Un README por aplicación con una descripción breve y las instrucciones de ejecución.
- Opcional: un video corto (máx. 3 minutos) demostrando las tres apps.

06 Criterios de evaluación

Cada aplicación se evalúa sobre 20 puntos con la siguiente distribución. La calificación final es el promedio de las tres aplicaciones.

Criterio	Descripción	Puntos
Funcionalidad	Cumple todos los requisitos mínimos del enunciado.	10
Estado y navegación	Uso correcto de setState, navegación con paso de datos y validación / Timer.	5
Interfaz	Diseño claro y ordenado: layout, espaciado y legibilidad.	3
Casos límite	Manejo de entradas vacías o inválidas y del estado vacío.	5
Código	Organización y legibilidad: nombres, estructura, dart format.	5
Total por aplicación		30

NOTA

Los retos opcionales se acreditan como bonus (10 puntos) solo si los requisitos mínimos de la aplicación están completos. Una app que se cierra ante una entrada inválida no obtiene el puntaje de «Casos límite».

07 Forma de entrega

- Sube tu solución a un repositorio de GitHub personal (público o con acceso al docente) y envía el enlace en un archivo de texto a la plataforma virtual de la universidad.
- Verifica que cada proyecto compile con `flutter run` antes de entregar.